

Описание проекта Termit

Документ для заполнения форм, регистрации и технической документации.

Версия системы: 0.1.0 | стек: Python 3.9+, FastAPI

1. Наименование и назначение системы

Termit — open-source AI-оркестратор для написания кода и автоматизации задач на локальном компьютере и в веб-среде.

Система объединяет:

- чат с LLM и маршрутизацию моделей;
- выполнение задач по коду (plan → execute → verify → report);
- безопасные локальные инструменты (файлы, shell);
- веб-автоматизацию;
- профили агентов с фоновым выполнением;
- мониторинг KPI, eval-набор и ops-проверки.

Целевая аудитория: разработчики и команды, которым нужна единая платформа для AI-assisted coding с контролем безопасности, квот и наблюдаемости.

2. Функциональные возможности

1. Чат с LLM — синхронные и streaming-ответы (SSE), память сессий, экспорт истории (markdown/txt/json), кэширование ответов, compaction контекста, RAG по кодовой базе workspace.
2. Маршрутизация моделей — выбор primary/fallback моделей по типу задачи (coding, review, debug, explain, general), оценка сложности запроса, circuit breaker и retry при сбоях провайдеров.
3. Жизненный цикл задач — plan → execute → verify → report; режимы auto и guided; retry по классам ошибок; события и structured report; хранение в SQLite или in-memory.
4. Локальные инструменты — list/read файлов в sandbox workspace, execute shell-команд с политикой safe / confirm / blocked, dry-run, audit trail.
5. Веб-автоматизация — HTTP-fetch страниц, извлечение title/links/snapshot, детекция блокиров (login, captcha, 403), anti-loop через max_steps.
6. Агенты — CRUD профилей, синхронный и фоновый запуск (очередь + worker threads), прокси инструментов с проверкой enabled_tools, online automation.
7. Multi-agent orchestration — pipeline planner → executor → verifier → task runner с phased report и duration_ms.
8. Retrieval по кодовой базе — keyword-индекс по workspace, chunking, scoring, фильтр по path_prefix, reindex и stats.
9. Auth и квоты — API keys, RBAC (viewer/operator/admin), daily quota per key, team quotas через TeamWorkspaceService.
10. Observability — runtime telemetry, KPI snapshots (JSONL), daily/executive reports, Slack text/payload, eval suite (24 сценария), feedback, ops readiness и incident drill.

11. Локальный runtime — статус Ollama/OpenAI-compatible провайдеров, список локальных моделей, pull моделей в Ollama.

12. Web UI — минимальный веб-интерфейс для чата и управления (/).

3. Логика работы (бизнес-процессы)

3.1. Обработка chat-запроса

Клиент → POST /api/chat (или /api/chat/stream)

- AuthQuotaMiddleware (ключ, RBAC, квота)
- ChatService: загрузка истории сессии
- ContextCompactor (если превышены лимиты)
- CodeRetrievalService (если use_retrieval=true)
- ModelRouter: выбор primary/fallback модели
- Provider (Ollama / OpenAI-compat) + circuit breaker + retry
- TelemetryStore: latency, cache, cost proxy, quality signals
- Ответ (JSON) или SSE stream (meta, token, error, done)

3.2. Выполнение задачи

POST /api/tasks → TaskService

- analyze_input
- inspect_workspace
- read_readme
- compose_report
- verify (tests, expectations)
- completed / failed / cancelled
- retry по классу ошибки (planning_error, verification_error, external_error)
- события в TaskStore (SQLite или memory)

3.3. Выполнение shell-команды

POST /api/tools/execute

- parse command
- classify risk: safe | confirm | blocked
- path/command policy checks
- execute (или dry-run)
- audit log entry

3.4. Запуск агента

POST /api/agents/{id}/run (или /runs для фонового режима)

- загрузка профиля из AgentRegistryStore
- проверка enabled_tools и online policy
- ChatService + ToolingService + BrowserWorkflowService
- сохранение состояния в AgentRunStore

3.5. Сбор KPI

Runtime events → TelemetryStore (in-memory)

- POST /api/metrics/snapshot → JSONL file
- GET daily-report / executive-summary / slack payload
- CLI scripts/metrics_report.py

4. Архитектура системы

Слой	Назначение
API Layer	HTTP-эндпоинты, валидация, SSE-streaming
Orchestration Layer	Plan/execute/verify/report task loop, multi-agent pipeline
Provider Layer	Ollama, OpenAI-compatible adapters, routing policies
Tools Layer	File, shell, browser, project-aware utilities
Memory Layer	Session memory, task history, agent runs
Observability Layer	Traces, metrics, eval, feedback, ops

Точка входа: `app/main.py` — FastAPI app, middleware (CORS, security headers, auth/quota), подключение роутеров.

Конфигурация: `app/core/config.py` — переменные окружения.

DI: `app/state.py` — фабрики singleton-сервисов.

5. Состав программных модулей

Модуль	Путь	Назначение
Точка входа	<code>app/main.py</code>	FastAPI app, middleware, роутеры
Конфигурация	<code>app/core/config.py</code>	Env-настройки (models, paths, auth, cache, retry)
Auth	<code>app/core/auth.py</code>	Извлечение API key, public paths
RBAC	<code>app/core/rbac.py</code>	Роли viewer/operator/admin, матрица доступа
Auth middleware	<code>app/middleware/auth_quota.py</code>	Ключи, RBAC, daily quota
Security headers	<code>app/middleware/security_headers.py</code>	HTTP security headers
Chat	<code>app/services/chat_service.py</code>	Чат, streaming, memory, cache, retrieval
Model router	<code>app/services/model_router.py</code>	Маршрутизация по task type и complexity
Context compaction	<code>app/services/context_compaction.py</code>	Сжатие истории с summary
Code retrieval	<code>app/services/code_retrieval_service.py</code>	Keyword-индекс workspace

Модуль	Путь	Назначение
Circuit breaker	app/services/provider_circuit_breaker.py	Защита от каскадных сбоев провайдеров
Tasks	app/services/task_service.py	Оркестрация задач
Task store	app/services/task_store.py, sqlite_task_store.py	In-memory / SQLite persistence
Tools	app/services/tooling_service.py	Файлы, shell, audit
Browser	app/services/browser_workflow_service.py	Веб-автоматизация
Agents	app/services/agent_service.py	Профили, runs, background queue
Agent stores	agent_registry_store.py, agent_run_store.py	JSON profiles, SQLite runs
Orchestration	app/services/multi_agent_orchestrator.py	Multi-agent pipeline
Telemetry	app/services/telemetry_store.py	Runtime-метрики
Metrics snapshots	app/services/metrics_snapshot_store.py	JSONL snapshots, reports, Slack
Quota	app/services/quota_store.py	SQLite учёт запросов
Teams	app/services/team_workspace_service.py	Team quotas и workspace
Eval	app/services/eval_service.py	24 benchmark-сценария
Ops	app/services/ops_service.py	Readiness, incident drill, quota admin
Local runtime	app/services/local_runtime_service.py	Ollama status, models, pull
Providers	app/services/providers/	OllamaProvider, OpenAICompatProvider
Web UI	app/web/routes.py	Jinja2 templates

6. API-эндпоинты

Группа	Эндпоинты	Назначение
Chat	POST /api/chat, POST /api/chat/stream	Диалог, SSE streaming
Sessions	GET/DELETE /api/sessions/{id}, GET .../export	Память, экспорт

Группа	Эндпоинты	Назначение
Providers	GET /api/providers/status	Статус провайдеров
Tasks	POST/GET /api/tasks, GET/DELETE /api/tasks/{id}, GET .../events	CRUD задач
Tools	GET /api/tools, POST /api/tools/list-files, read-file, execute, GET .../audit	Файлы, shell, audit
Automation	POST /api/automation/web	Веб-автоматизация
Agents	GET/POST /api/agents, POST /api/agents/{id}/run, GET/POST /api/agents/runs/*	Профили, runs
Orchestration	POST /api/orchestration/run	Multi-agent pipeline
Retrieval	POST /api/retrieval/search, POST /reindex, GET /stats	Поиск по коду
Metrics	GET /api/metrics, POST /snapshot, GET /trend, /daily-report, /executive-summary, /slack	KPI
Eval	GET /api/eval/scenarios, POST /run, POST /run-suite	Benchmark
Ops	GET /api/ops/readiness, POST /incident-drill, GET/POST /quota/*	Эксплуатация
Usage	GET /api/usage	Квота текущего ключа
Teams	GET /api/teams/*	Team workspace
Local	GET /api/local/status, GET /models, POST /models/pull	Локальные модели
Feedback	POST /api/feedback	Обратная связь
Health	GET /health	Health check
Web	GET /	Web UI

7. Скрипты

7.1. scripts/metrics_report.py

CLI для KPI через HTTP API Termit.

Режимы (--mode):

Режим	HTTP	Описание
snapshot	POST /api/metrics/snapshot	Захват snapshot метрик
daily-report	GET /api/metrics/daily-report	Дневной отчёт
executive-summary	GET /api/metrics/executive-summary	Executive KPI summary
slack-summary	GET .../executive-summary/slack	Текст для Slack
slack-payload	GET .../executive-summary/slack/payload	JSON для Incoming Webhook

Параметры: --base-url (default: http://127.0.0.1:8765), --days (default: 7), --limit (default: 200).

Пример cron:

```
5 * * * * cd /path/to/Termit && python3 scripts/metrics_report.py --mode snapshot
```

7.2. scripts/export_kpi_snapshot.py

Экспорт текущих метрик в JSON.

Параметры:

- --base-url — URL Termit API (default: http://localhost:8765)
- --api-key — ключ при TERMIT_AUTH_ENABLED=true
- --capture — POST snapshot перед экспортом
- --output — файл или - для stdout

Пример:

```
python3 scripts/export_kpi_snapshot.py --capture --output kpi.json
```

7.3. scripts/run_incident_drill.py

Запуск ops-проверок через API.

Режимы:

- --readiness-only → GET /api/ops/readiness
- по умолчанию → POST /api/ops/incident-drill (требуется admin key)

Параметры: --base-url, --api-key, --output.

Exit code: 0 при status = ready или degraded, иначе 1.

Пример:

```
python3 scripts/run_incident_drill.py --api-key "$TERMIT_ADMIN_KEY"
```

8. Используемые технологии

Категория	Технологии
Язык	Python 3.9+
Web framework	FastAPI, Uvicorn, Starlette
HTTP client	httpx
Шаблоны	Jinja2
База данных	SQLite
Файловое хранение	JSON, JSONL
LLM провайдеры	Ollama, OpenAI-compatible API (vLLM, TGI, LM Studio, OpenRouter)
Контейнеризация	Docker, Docker Compose
Тестирование	unittest (89+ тестов)

9. Хранение данных

Компонент	Backend	Переменная / путь
Session memory	SQLite / memory	TERMIT_MEMORY_SQLITE_PATH
Task history	SQLite / memory	TERMIT_TASK_SQLITE_PATH
API quotas	SQLite	TERMIT_QUOTA_SQLITE_PATH
Response cache	memory / SQLite	TERMIT_RESPONSE_CACHE_*
Agent profiles	JSON	TERMIT_AGENT_REGISTRY_FILE
Agent runs	SQLite	agent run store
Metrics snapshots	JSONL	TERMIT_METRICS_SNAPSHOT_FILE
Eval reports	JSONL	TERMIT_EVAL_REPORT_FILE
Feedback	JSONL	TERMIT_FEEDBACK_FILE

10. Безопасность

- API keys — constant-time сравнение (`secrets.compare_digest`)
- RBAC — роли viewer / operator / admin с матрицей доступа по method+path
- Quotas — daily limit per key, team quotas
- Sandbox — file/shell ограничены workspace root, защита от path traversal

- Command policy — safe (auto), confirm (explicit flag), blocked (deny)
- Audit trail — лог всех tool-вызовов (GET /api/tools/audit, admin only)
- Security headers — middleware для HTTP-заголовков
- Публичные пути — /health, /docs, /api/ops/readiness без auth

11. Тестирование

Каталог: tests/ — 89+ unit/integration тестов.

Покрытие:

- chat, model router, context compaction, code retrieval
- task lifecycle, tasks API e2e (10 сценариев)
- tools safety, auth/quota, RBAC
- browser automation (8 online-сценариев)
- eval, metrics, Slack payload, agents, ops
- SQLite stores, response cache

Запуск:

```
python -m unittest discover -s tests -v
```

12. Развёртывание

Локально:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --host 0.0.0.0 --port 8765 --reload
```

Docker:

```
docker compose up --build
```

Документация: HOSTED_DEPLOYMENT.md, Dockerfile, docker-compose.yml.

13. Связанная документация

Файл	Содержание
README.md	Quick start, API overview
MVP_ARCHITECTURE.md	Архитектура MVP
ROADMAP_90_DAYS.md	90-дневный roadmap
SPRINT_BACKLOG.md	6 спринтов с DoD
USER_JOURNEYS.md	7 пользовательских сценариев
TOOL_SAFETY_POLICY.md	Политика безопасности инструментов

Файл	Содержание
EVAL_SUITE.md	24 eval-сценария
TASK_EXECUTION_CONTRACT.md	Контракт выполнения задач
KPI_DASHBOARD_SPEC.md	Спецификация KPI-дашборда
INCIDENT_RUNBOOK.md	Runbook инцидентов
BETA_ONBOARDING.md	Онбординг beta
BENCHMARK_SPEC.md	Спецификация benchmark
OBSERVABILITY_CHECKLIST.md	Чеклист observability
RELEASE_CHECKLIST.md	Чеклист релиза
HOSTED_DEPLOYMENT.md	Hosted deployment

14. Краткие формулировки для форм

~500 символов

Termit — AI-оркестратор (FastAPI, Python) для coding и автоматизации. Чат с LLM (Ollama, OpenAI-compatible), streaming, память сессий, retrieval по коду, задачи plan/execute/verify/report, безопасные shell/file tools, веб-автоматизация, агенты, RBAC и квоты, KPI/eval/ops. Скрипты: metrics_report, export_kpi_snapshot, run_incident_drill.

~1000 символов

Termit — open-source платформа AI-assisted coding на FastAPI (Python 3.9+). Маршрутизирует запросы к локальным LLM, поддерживает chat/streaming, compaction контекста, RAG по workspace, жизненный цикл задач с retry и structured report. Локальные инструменты: файлы и shell с политикой safe/confirm/blocked и audit. Веб-автоматизация с anti-loop. Профили агентов, multi-agent orchestration, auth (API keys), RBAC, квоты. Observability: telemetry, KPI snapshots, eval (24 сценария), incident drill. Хранение: SQLite, JSON/JSONL. CLI: metrics_report.py, export_kpi_snapshot.py, run_incident_drill.py. 89+ тестов, Docker deploy.

~2000 символов

Termit — AI-оркестратор для написания кода и автоматизации. Архитектура: API → Orchestration → Providers → Tools → Memory → Observability.

Функции: (1) Chat с streaming, session memory, response cache, context compaction, code retrieval; (2) Model routing по task type с fallback, circuit breaker, retry; (3) Task lifecycle plan→execute→verify→report, SQLite/in-memory store; (4) Tools: list/read files, execute_command с risk policy и audit; (5) Browser automation: fetch, evidence, blocker detection; (6) Agents: profiles, sync/background runs; (7) Multi-agent orchestration; (8) Auth/RBAC/quotas; (9) Metrics, eval suite, ops readiness.

Технологии: Python 3.9+, FastAPI, httpx, Jinja2, SQLite, Docker. Провайдеры: Ollama, OpenAI-compatible.

Скрипты: metrics_report.py (KPI snapshot/reports/Slack), export_kpi_snapshot.py (JSON export), run_incident_drill.py (readiness/drill).

Безопасность: API keys, RBAC (viewer/operator/admin), sandbox, command allowlist/blocklist, audit trail.

Один абзац (универсальный)

Termit — AI-оркестратор для написания кода и автоматизации задач на базе FastAPI. Система маршрутизирует запросы к локальным LLM (Ollama, OpenAI-compatible), поддерживает чат с памятью сессий, streaming, retrieval по кодовой базе, жизненный цикл задач (plan/execute/verify/report), безопасное выполнение shell-команд и веб-автоматизацию, профили агентов с фоновым выполнением, multi-agent orchestration, авторизацию с RBAC и квотами, telemetry/KPI-отчёты и eval-набор из 24 сценариев. Вспомогательные скрипты автоматизируют snapshot метрик, экспорт KPI и incident drill.

15. Структура для Word/PDF

1. Титульный лист — Termit v0.1.0, AI coding orchestrator
2. Аннотация — раздел 14 (~500 символов)
3. Назначение системы — раздел 1
4. Функциональные возможности — раздел 2
5. Алгоритмы и логика работы — раздел 3
6. Архитектура — раздел 4
7. Состав модулей — раздел 5
8. API-интерфейсы — раздел 6
9. Скрипты — раздел 7
10. Технологический стек — раздел 8
11. Хранение данных — раздел 9
12. Безопасность — раздел 10
13. Тестирование — раздел 11
14. Развёртывание — раздел 12
15. Приложения — ссылки на docs (раздел 13)